System Design — URL Shortener (miniurl.com)

> Week 1 of the System Design track. Format: reasoning + tradeoffs + ASCII diagrams. The point isn't the boxes — it's defending every choice against the obvious alternative.

---

1. Requirements

Functional - Shorten a long URL → return a short URL. - Update / delete an existing short URL. - Track click analytics (how many times each short URL was visited).

Non-functional - Read-heavy (redirects ≫ creates). - Globally available, low latency worldwide. - Product limits (assumptions, stated not assumed silently): 5 URLs/user/day, links expire after 30 days.

Explicitly out of scope: the "5 edits share the create counter" rule — product policy, not architecture. Knowing what to ignore is a skill.

---

2. Back-of-envelope estimates

Assume **10k active creators**, 100:1 read:write ratio.

Metric	Calc	Result
Writes/day (creates)	10k users × 5 URLs	50,000/day
Write QPS	50,000 / 86,400	**~0.6/sec (trivial)**
Reads/day (redirects)	50,000 × 100	5,000,000/day
Read QPS (avg)	5M / 86,400	~58/sec
Read QPS (peak ~5×)		**~200–300/sec**
Live storage	50k/day × 30-day expiry × ~500 B/row	**~750 MB (tiny)**

What the numbers tell us (the actual conclusion): The DB is NOT the problem — writes trivial, storage sub-gigabyte. This is a **read-latency + unique-code-generation problem**, not a database-scaling problem.

---

3. Key design decisions (each defended vs the obvious alternative)

3a. Redirect status code — 302, not 301

- **301 (permanent):** cached forever by browser/CDN → fastest, BUT the click never hits our server again → **analytics counter never fires**, and edited/deleted links keep resolving to stale targets.
- **302 (temporary):** click returns to us each time → keeps analytics + editability.
- **Choice: 302.** Trade some redirect speed to preserve two functional requirements.

3b. Short-code generation — counter + base62 (not hash, not random)

Strategy	Collision?	Cost
Hash long URL (MD5/SHA) → base62, first 7	yes	check + rehash per write
Random (nanoid) → base62	yes	check + retry per write
Counter + base62 encode	**none by design**	needs a global counter
- **Chosen: counter + base62.** Unique by construction, zero collision checks.		
- **Downside → fix:** global counter is a bottleneck/SPOF → hand each app server a		
range of IDs (server A: 1–1000, B: 1001–2000) or use **Snowflake-style IDs**.		
- **Length:** base62, 7 chars = $62^7 \approx$ **3.5 trillion** codes. Plenty.		

3c. Click counter — async via queue (not inline write)

- **Obvious:** `UPDATE count = count+1` on every redirect → turns the hot read path into a write on every click = **write amplification**, kills read performance.
- **Better:** fire-and-forget the click event to a **queue** (SQS) / **stream** (Kinesis/Kafka at scale); redirect user immediately; aggregate counts async.
- **Batching fix:** consumer aggregates per-short-code in memory, flushes on **count OR time** (e.g. every 100 hits *or* every 10s) — the time window prevents low-traffic URLs from being stuck un-flushed. Cuts writes ~100:1.
- Count is eventually consistent (slightly stale) — acceptable, nobody needs it live.

3d. Global low latency — CDN edge caching

- CloudFront / Cloudflare caches the short→long mapping at edge PoPs.
- Cache invalidation needed on update/delete.

---

4. API design

```
POST   /urls                → shorten. body {longUrl} → {shortCode, shortUrl}
GET    /{shortCode}         → REDIRECT 302 (hot path, ~300 QPS, sub-10ms) — not JSON
GET    /urls/{shortCode}    → fetch one (edit view, cold)
GET    /urls?userId=&page=  → list user's urls, paginated (cold, dashboard)
PATCH  /urls/{shortCode}    → update target {longUrl}
DELETE /urls/{shortCode}    → delete
```

Hot vs cold split matters: the redirect has a sub-10ms budget; the dashboard GETs don't.

5. Data model — NoSQL (key-value, e.g. DynamoDB)

Why NoSQL, not SQL (the defense): access pattern is lookup-by-single-key (shortCode) → one row, no joins, no cross-row transactions, tiny records. That's a key-value store's sweet spot. SQL's strengths (joins, referential integrity, multi-row ACID) buy nothing here. Pick DynamoDB/Redis-backed KV.

The principle: *SQL models by entity; NoSQL models by ACCESS PATTERN.* Shape data so the hot query is a direct key hit.

Trap avoided: do NOT embed URLs inside the user document. The redirect looks up by shortCode with no user context 300×/sec — nesting URLs under users would force an O(n) scan across all user docs. URL is its OWN item, keyed by shortCode.

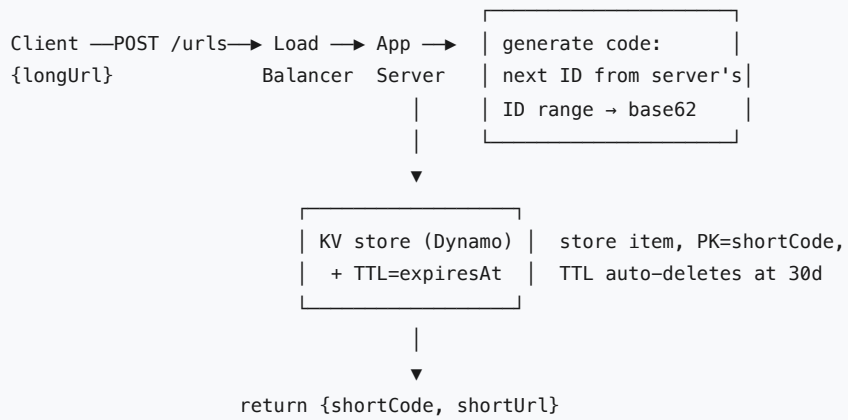
```
URL item (hot — redirect):
  PK: shortCode      ← O(1), 300 QPS, sub-10ms
  longUrl
  userId             ← owner (plain value)
  clickCount
  createdAt
  expiresAt          ← DynamoDB TTL auto-deletes at 30 days (no cron)

GSI on userId (cold — "list my URLs"):
  PK: userId,  SK: createdAt   → user's links, newest first, paginated
```

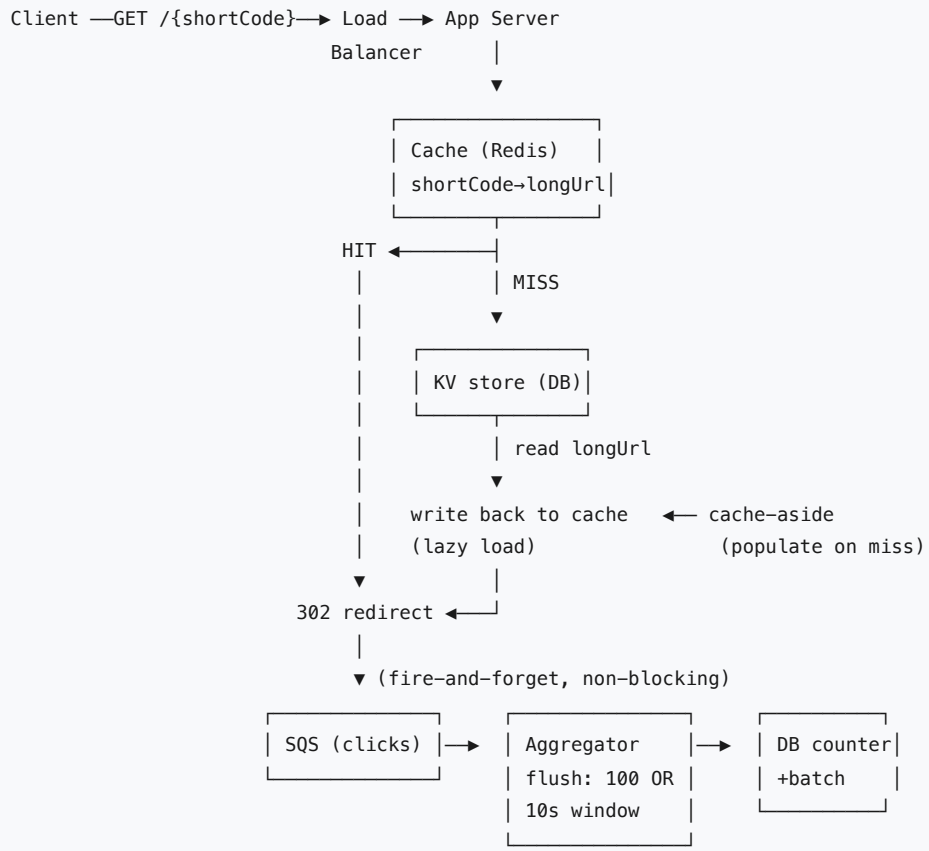
Two access patterns → two keys. Embed what you read together; split what you read apart.

6. High-level architecture

Write path — create a short URL (cold, ~0.6 QPS)



Read path — redirect (HOT, ~300 QPS, sub-10ms)



Key points to say out loud: - Redirect is **302** (not 301) — must return to server so the click counter fires. - **Cache-aside**: check Redis → miss → read DB → populate Redis → redirect. Next hit is O(1) cache. - Counter is **fire-and-forget** to SQS off the hot path → redirect never waits on analytics. - Read is served from cache ~99% of the time (hot links dominate) → DB barely touched.

7. Bottlenecks & tradeoffs

Bottleneck	Trigger	Fix
Global ID counter	mass URL creation / counter server dies	**Key-range allocation** — each app server owns a block (1–1k, 1k–2k…); no per-write coordination. Or Snowflake IDs.
Hot key (viral link)	millions of hits to ONE shortCode → cache read-throughput limit	Sharding does NOT help (it's one key). **Replicate** the key across cache replicas + load-balance, or **app-local cache** (in-memory on each app server, no network hop).
Cache stampede / thundering herd	cache restart or mass TTL expiry → all misses hit DB at once	**Request coalescing (single-flight)** — first miss loads, rest wait for it. **TTL jitter** so keys don't expire together.
CAP under partition	network split	**AP over CP** — favor availability. A slightly-stale redirect beats a redirect that errors. Redirects must stay up.

Overall shape of the system: reads dominate → cache-aside + 302; writes trivial → any KV store; analytics off the hot path → async queue. The "hard" parts are all about read latency and a single viral key, never about scaling writes or storage.

Status: COMPLETE ✅

All 7 sections done. Ready to export to PDF.

Vocabulary surfaced (also in /VOCAB_MAP.md)

write amplification · fire-and-forget · message queue vs stream · eventual consistency · base62 encoding · distributed ID generation · Snowflake · key-range allocation · 301 vs 302 · cache invalidation · batching / flush interval / aggregation window